Apellidos y nombre: ..... Grupo:.....

Examen Final (2ª parte)

- Duración del examen: 2 horas.
- La solución se tiene que escribir en el espacio reservado para ello en el propio enunciado.
- No podéis utilizar calculadora, móvil, apuntes, etc.
- La solución del examen se publicará en el Racó de la FIB y en Atenea mañana.
- La revisión será el **19** de **Junio** a las **11:00** en el aula **C6-E101** Las notas definitivas se publicarán en el Racó de la FIB al día siguiente.

Ejercicio 1 **(1,5 puntos)**

Vamos a analizar diferentes aspectos de la ejecución, en el computador SISC von Neumann, del siguiente fragmento de código:

```
MOVI R7, -2
SHA  R0, R1, R7
```

a) Si antes de comenzar a ejecutar la primera instrucción el estado del computador es el siguiente:
PC = 0x348E, R0 = 0x348E, R1 = 0x82A5, R7 = 0x0002
¿Completad el estado del computador que queda después de ejecutada la segunda instrucción? (**0,2 puntos**)

PC = 0x	, R0 = 0x	, R1 = 0x	, R7= 0x	,
MEM_b[0x348E] = 0x		, MEM_b[0x348F] = 0x		.

b) Sea Nc el número de ciclos que tarda en ejecutarse el código. ¿Cuánto vale Nc? (**0,1 puntos**) Nc =

c) Completad las 4 filas de la siguiente tabla que hacen referencia a los ciclos de ejecución 3, 4, 5 y 6 (los ciclos de ejecución se numeran desde el 1 al Nc). Podéis ver los mnemotécnicos de salida de cada Nodo en el grafo de la chuleta. En la columna Acciones usad el lenguaje de transferencia de registros de la documentación de la asignatura (o una notación igual de precisa). Recordad que en la Palabra de control compactada solo hay que expresar el valor de los campos que son significativos en ese ciclo (no hay que indicar los campos que valen x ni las señales de permiso de modificación del estado que valen 0). Completad esta tabla para el **caso general** del código **`MOVI Rd,N8`** y **`SHA Rd,Ra,Rb`**. (**0,4 puntos**)

Ciclo	**Nodo** Mnemo Salida	**Acciones**	**Palabra de control compactada**
3			
4			
5			
6			

d) Completad la siguiente tabla, que es una extensión de la anterior con nuevas columnas y solo dos filas (ciclos 5 y 6), aunque ahora las filas son más estrechas. Repetimos las dos primeras columnas para que quede más claro. En la columna Instrucción en IR poned "No se sabe" cuando este sea el caso. La palabra de control es la que genera el SISC CONTROL UNIT y por lo tanto solo tenéis que poner x en los campos en los que no podáis saber su valor del campo porque no se sabe cómo se han implementado las x en la ROM_OUT. Para rellenar esta tabla usad el **código concreto** **`MOVI R7,-2`** y **`SHA R0,R1,R7`**. (**0,4 puntos**)

Ciclo	**Nodo** Mnemo Salida	**Instrucción en IR** (en ensamblador)	**Palabra de Control**																		
			@A	@B	Pc/Rx	Ry/N	OP	F	P/I/L/A	@D	WrD	Wr-Out	Rd-In	Wr-Mem	Ldlr	LdPc	Byte	Alu/R@	R@/Pc	N (hexa)	ADDR-IO (hexa)
5																					
6																					

e) Completa siguiente tabla, que puede considerarse una continuación de las dos anteriores pero ahora con solo las filas de los ciclos 3 y 6 pero con nuevas columnas: la dirección (en decimal) de la ROM_OUT que se accede en cada ciclo y los 24 bits de su contenido (poned x siempre que un bit pueda valer tanto 0 como 1). (**0,4 puntos**)

Ciclo	**Nodo** Mnemo Salida	**@ROM**	Bnz	Bz	WrMem	RdIn	WrOut	WrD	Ldlr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/L/A1	P/I/L/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0
3																										
6																										

Ejercicio 2 **(1,3 puntos)**
Completad el diseño del SISC Von Neumann para que pueda ejecutar, además de las 25 instrucciones originales SISA, la nueva instrucción

```
SHAI Rd, Ra, N5
```

que al ejecutarse modifica el estado del computador de la misma forma que lo hace la instrucción SHA pero siendo el segundo operando un inmediato de 5 bits que codifica un número entero en complemento a 2. Con esta nueva instrucción podemos hacer lo mismo que en el SISA original requiere dos instrucciones (como las dos del fragmento de código del *Ejercicio 1*).

La nueva instrucción se tiene que poder ejecutar sin efectuar ningún cambio en el computador excepto el contenido de la ROM_Q+ y el de la ROM_OUT. La nueva instrucción se tiene que ejecutar en el mínimo número de ciclos. La ejecución de las 25 instrucciones SISA originales se hace exactamente igual que antes de introducir la nueva instrucción. Se pide:

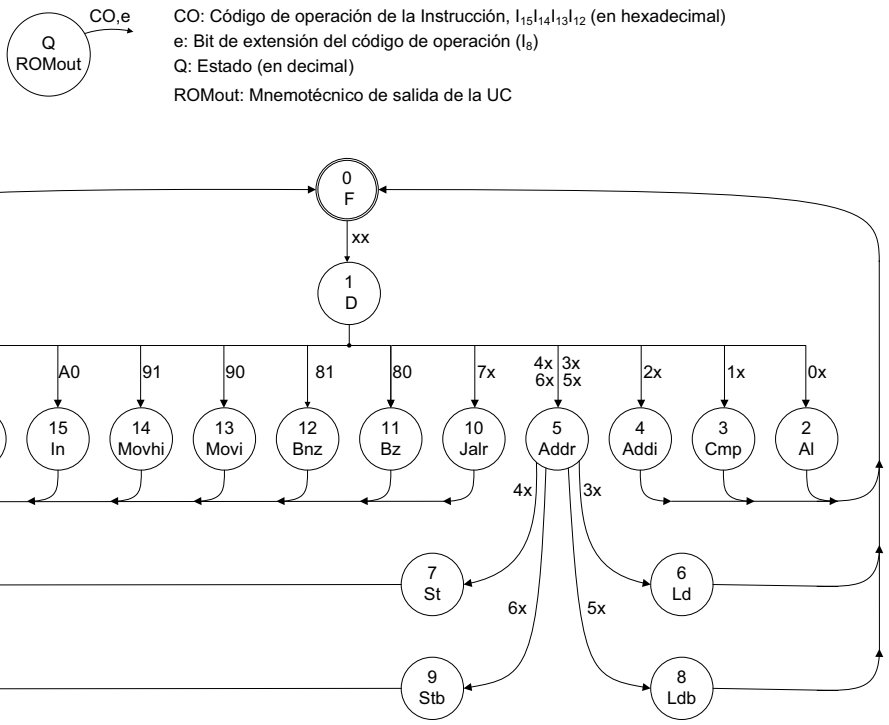
a) Proponed el formato y la codificación en binario de la nueva instrucción (cada bit puede valer `0`, `1`, `a`, `b`, `d`, `n`, `x`) y escribid la semántica usando un fragmento de código en un lenguaje de alto nivel tipo C, como el empleado para esto en la documentación. (**0,1+0,2 puntos**)

`Codificación:`

`Sintaxis:` `SHAI Rd, Ra, N5`

`Semántica:`

b) Modificad el grafo de estados de la unidad de control (añadiendo nodos y arcos y/o modificando los ya existentes) dibujando sobre y/o al lado del siguiente grafo original. (**0,2 puntos**)



c) Completad la tabla usando una fila para cada nuevo nodo que hayáis añadido al grafo (no son más de 2 nodos). (**0,3 puntos**)

Nodo		Acciones	Palabra de control compactada
Estado (decimal)	Mnemo salida		

d) Indicad la dirección o las direcciones (en binario, con x cuando sea posible para referirnos a más de una dirección) de la ROM_Q+ y su conten ido (en hexadecimal) para implementar correctamente el arco que va del nodo/estado 1 (con mnemotécnico de salida D) al primer nuevo nodo usado para implementar la nueva instrucción. (**0,2 puntos**)

Arco del nodo 1 (D) al nodo	**Dirección o direcciones** (en binario)	**Contenido** (en hexa)

e) Completad (poniendo 0, 1 o x en cada bit) la siguiente tabla que especifica, mediante una fila por cada nuevo nodo añadido al grafo de estados de la unidad de control (no son más de 2 nodos), la dirección (en decimal) de la ROM_OUT y su contenido para que se ejecute correctamente la nueva instrucción, poniendo el máximo número de x posibles. En la columna de la derecha poned elmnemotécnico de salida que habís usado para nombrar cada nuevo nodo en el grafo del apartado b). (**0,3 puntos**)

@ROM	Bnz	Bz	WrMem	RdIn	WrOut	WrD	Ldlr	Byte	R@/Pc	Alu/R@	Pc/Rx	Ry/N	P/I/LL/A1	P/I/LL/A0	OP1	OP0	MxN1	MxN0	MxF	F2	F1	F0	Mx@D1	Mx@D0	**Nodo (Mnemo Salida)**

Ejercicio 3 **(1,2 puntos)**

Completar el siguiente código ensamblador SISA que entra por el teclado un dato de 16 bits, X, que es interpretado como número natural, lo eleva al cuadrado multiplicandolo por si mismo y finalmente imprime el resultado. Solo calcula e imprime los 16 bits de menor peso del resultado y no detecta si el resultado correcto para naturales es representable o no en 16 bits.

Para multiplicar usa un algoritmo parecido al de la práctica 5 del Lab de IC "con terminación temprana" (que termina cuando lo indica el multiplicador). Recordad que en cada iteración del algoritmo de la práctica se multiplica un bit del multiplicador por el multiplicando y se acumula el resultado; además se efectúan desplazamientos del multiplicando y del multiplicador. Se itera a lo sumo 16 veces, pero si el multiplicador tiene ceros en los bits de más peso termina en menos iteraciones que si no los tiene.

La diferencia entre el algoritmo de la práctica y el de este ejercicio es que este puede terminar antes de las 16 iteraciones tanto por que lo indique el multiplicando como el multiplicador.

El uso de registros en el código es el siguiente:

- R7 se usa siempre que haga falta un registro temporal para almacenar un dato que se usará en la siguiente instrucción.
- R1 es donde se almacena el multiplicando leído del teclado, que se modificará en cada iteración.
- R2 es inicialmente una copia de R1, pero se va modificando en cada iteración haciendo el papel de multiplicador.
- R3 y R4 se usan para almacenar valores constantes que no se modifican en ninguna de las iteraciones del bucle.
- R0 hace de registro acumulador.

```
.text
Key:    |      | R7, |      |
        |      | R7, |      |
        |      | R1, |      |
        | ADDI | R2, |      |
        | M    |     | , 1  |
        | M    | R4, |      |
        | M    | R0, |      |
Loop:   |      | R7, |      |
        | B    | R7, |      |
        | A    | R0, |      |
EndIf:  |      | R1, |      |
        |      | R1, |      |
        |      | R2, |      |
        |      | R2, |      |
Print:  |      | R7, |      |
        |      | R7, |      |
End:    |                   |
.end
```

a) Escribid el contenido de las cajas vacías para que el código sea el correcto en el lenguaje ensamblador SISA. Usad el nombre simbólico de los puertos del teclado (KEY-STATUS y KEY-DATA) y de la impresora (PRINT-STATUS y PRINT-DATA). Recordad que el acceso a los puertos de datos tiene un efecto lateral sobre los puertos de datos respectivos (**0,5 puntos**)

b) Si el dato entrado por el teclado es 0x004A:

¿Cuál es el resultado que se imprime? (**0,1 puntos**)

Resultado = 0x

¿Quién indica la finalización temprana en este caso, el multiplicando o el multiplicador? (**0,1 puntos**)

El ..... indica la finalización.

¿Cuántas iteraciones realiza el algoritmo (cada iteración va desde la etiqueta `Loop` a la `Print`)? (**0,1 puntos**)

Realiza ..... iteraciones.

Para los apartados siguientes suponed que los puertos de estado del teclado (KEY-STATUS) y de la impresora (PRINT-STATUS) contienen un 0x0001 cuando se ejecutan las instrucciones que los leen.

c) ¿Cuántas instrucciones se ejecutan cuando el dato entrado vale Xu=0x000F y cuantas para Xu=0x0F00? (**0,2 puntos**)

Para Xu=0x000F: instrucciones.
Para Xu=0x0F00: instrucciones.

d) ¿Cuánto vale x para que sea cierta la siguiente afirmación? "El código se ejecuta un x% más rápido cuando el dato entrado por el teclado es el 0x000F que cuando es el 0x0F00". Escribe una fórmula para x además de su valor. (**0,2 puntos**)

x = = ... %.